PDF Tools Hub

React · Vite · Axios · Java Spring Boot · Apache PDFBox · Apache POI

Organization	Team
Future Invo Solutions	Frontend: Gayathri | Backend: Ajay

Table of Contents

Table of Contents 2
1. Project Overview 3
1.1 Objectives 3
2. Feature Scope 3
3. Technology Stack 4
3.1 Frontend 4
3.2 Backend 4
3.3 Version Control & Tooling 4
4. System Architecture 4
5. Application Workflow 5
6. Backend Modules 5
6.1 Entry Point & CORS 6
6.2 PDF Conversion — PDFService & PDFController 6
6.3 PDF Editing — PDFEditingService & PDFEditingController 7
6.4 PDF Optimization — OptimizationService & OptimizationController 8
6.5 PDF Security — PDFSecurityService & PDFSecurityController 9
6.6 Exception Handling — CustomException & GlobalException 10
6.7 FileUtil — Shared Utility Class 10
7. Frontend Modules 11
7.1 Frontend Architecture 11
7.2 Component Structure 12
7.3 API Service Layer — api.js 13
7.4 FileUpload Component 14
7.5 PDF Tool Pages 14
7.6 User Interaction Flow 14
7.7 State Management 15
7.8 File Download Handling 16
7.9 Error Handling 16
8. Challenges & Solutions 16
9. Future Enhancements 17
10. Conclusion 17

1. Project Overview

Purpose

PDF Tools Hub is a full-stack web application that provides a browser-based interface for performing PDF operations. The React frontend handles user interactions and file uploads, while the Java Spring Boot backend processes files and returns results. The system covers four core categories: Conversion, Editing, Security, and Optimization.

1.1 Objectives

Frontend Objectives

- Design a simple, intuitive interface for uploading and managing PDF files
- Provide easy navigation between tools — merge, convert, compress, and more
- Enable seamless file upload and processed output download
- Display real-time feedback including loading states, success, and error messages

Backend Objectives

- Build RESTful APIs using Java Spring Boot for all PDF operations
- Implement PDF processing — merge, split, convert, lock/unlock, and compress
- Ensure efficient file handling via temporary storage and optimized processing
- Maintain clean layered architecture: Controller → Service → Utility

System & User Objectives

- Integrate frontend and backend through REST APIs with CORS-enabled communication
- Ensure fast, reliable processing and a scalable, modular design
- Reduce dependency on multiple tools by providing an all-in-one PDF solution
- Allow users to complete PDF tasks quickly without requiring technical knowledge

2. Feature Scope

Category	Features
PDF Conversion	PDF to Word | Word to PDF | Excel to PDF
PDF Editing	Merge PDF | Split PDF | Rotate PDF
PDF Security	Lock PDF (AES-256 encryption) | Unlock PDF
PDF Optimization	Compress PDF (JPEG image re-encoding)

3. Technology Stack

3.1 Frontend

Property	Details
React 18	Component-based UI library — functional components with hooks
Vite	Fast build tool — instant HMR, optimized production bundles
Axios	HTTP client — multipart uploads, progress tracking, response parsing
React Bootstrap	Pre-built components — Navbar, Grid, Button, Spinner, ProgressBar
React Router v6	Client-side routing — URL path to component mapping

3.2 Backend

Property	Details
Java + Spring Boot	Core REST API framework — Controllers, Services, @RestControllerAdvice
Apache PDFBox	PDF reading, manipulation, text extraction, image processing, encryption
Apache POI	Microsoft Office format handling — DOCX (XWPFDocument) and XLSX (XSSFWorkbook)
Maven	Dependency management and build automation

3.3 Version Control & Tooling

Property	Details
Git & GitHub	Source control, branching, and team collaboration
REST APIs	Stateless HTTP communication between frontend and backend
Temporary File Storage	UUID-named temp files managed by FileUtil utility class

4. System Architecture

The application follows a layered architecture with a clear separation between the React frontend and the Spring Boot backend. All communication occurs through REST APIs over HTTP.

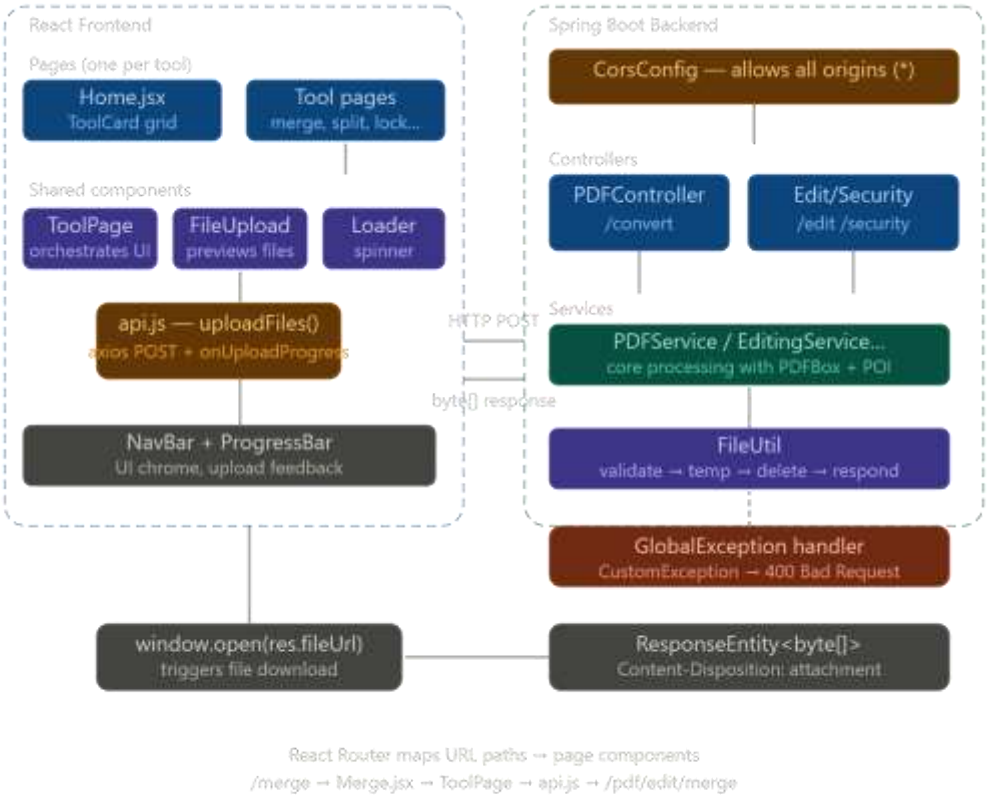

Figure 1: System Architecture — Full-Stack Overview

5. Application Workflow

The following steps describe the complete user journey from tool selection to file download.

1. User selects a tool — chooses a PDF operation from the homepage (e.g., Merge, Compress, Convert).
2. User uploads file(s) — selects one or more files via the file picker or drag-and-drop interface.
3. Frontend validates and sends request — React packages files as multipart/form-data and calls the backend via Axios.
4. Backend processes the file — Spring Boot service applies the operation using PDFBox or Apache POI.
5. Backend returns result — processed file is returned as a byte[] HTTP response with Content-Disposition: attachment.
6. User downloads the result — frontend converts the response to a Blob and triggers automatic browser download.

6. Backend Modules

The Spring Boot backend is organized into four functional domains, each with a dedicated Controller and Service. A shared FileUtil handles all file I/O, and GlobalException provides centralized error handling.

6.1 Entry Point & CORS

PdftoolshubApplication bootstraps the Spring context via @SpringBootApplication. CorsConfig registers a global CORS policy using @Configuration, permitting requests from any origin (*) so the React frontend on any domain can communicate with the API without browser security blocks.

6.2 PDF Conversion — PDFService & PDFController

Description

PDFService implements all three format conversion operations using Apache PDFBox for PDF reading/generation and Apache POI for Microsoft Office format handling. PDFController exposes the three conversion endpoints and delegates all logic to the service.

PDFService — Conversion Logic

- PDF to Word: PDFTextStripper extracts text with setSortByPosition(true), written into XWPFDocument paragraphs
- Word to PDF: XWPFDocument reads .docx paragraphs, PDFBox generates pages with automatic overflow handling
- Excel to PDF: XSSFWorkbook iterates sheets and rows, each sheet rendered as a landscape PDF page

PDFController — Endpoints

Property	Details
POST /pdf/convert/pdf-to-word	Converts uploaded PDF to .docx — validates file not empty
POST /pdf/convert/word-to-pdf	Converts uploaded .docx to PDF — validates .docx extension
POST /pdf/convert/excel-to-pdf	Converts uploaded .xlsx to PDF — iterates all sheets

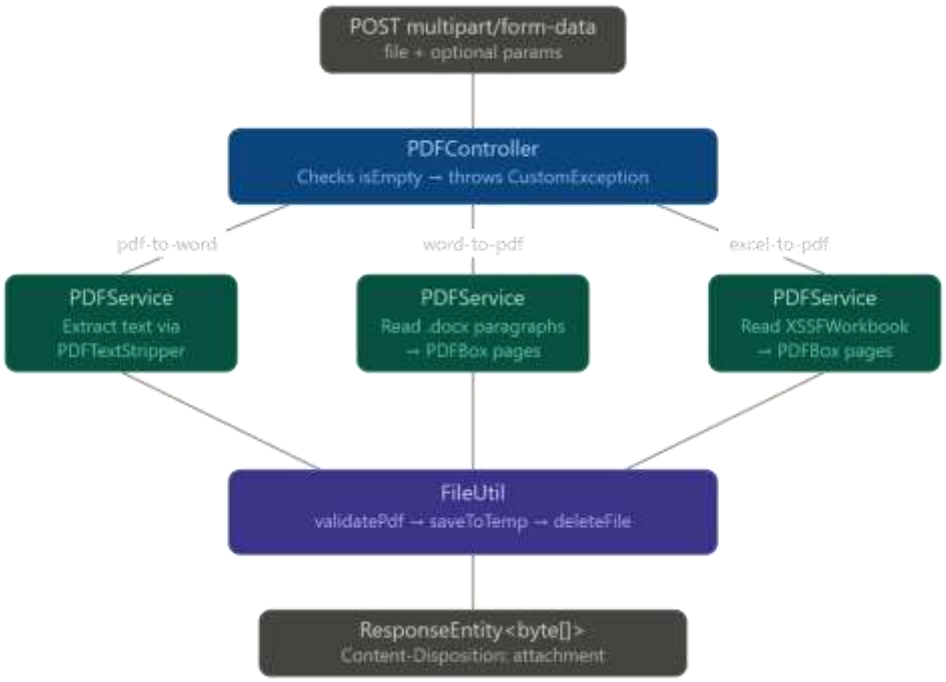

Figure 2: PDFController — Conversion Flow

6.3 PDF Editing — PDFEditingService & PDFEditingController

Description

PDFEditingService implements merge, split, and rotate operations using PDFBox utilities. Temporary files handle large uploads. All operations use try/finally to guarantee cleanup regardless of success or failure.

PDFEditingService — Core Logic

- Merge: PDFMergerUtility combines 2+ files in order; throws CustomException if fewer than 2 files provided
- Split: PDFBox Splitter extracts startPage–endPage range; endPage defaults to last page when -1
- Rotate: resolvePages() parses 'all' or '1,3,5' into Set<Integer>; rotation applied as (current + angle) % 360

PDFEditingController — Endpoints

Property	Details
POST /pdf/edit/merge	Accepts List<MultipartFile> — minimum 2 files required
POST /pdf/edit/split	Accepts file + startPage (default 1) + endPage (default -1)
POST /pdf/edit/rotate	Accepts file + angle (90/180/270) + pageNums string

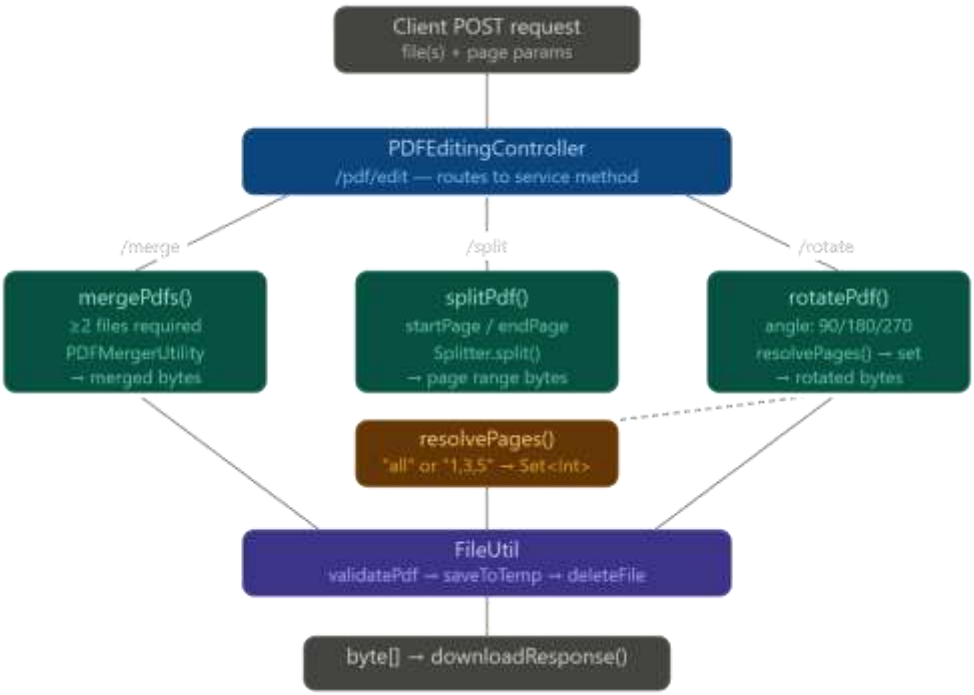

Figure 3: PDFEditingController — Editing Operations Flow

6.4 PDF Optimization — OptimizationService & OptimizationController

Description

OptimizationService reduces PDF file size by re-encoding embedded images as JPEG with reduced quality. It iterates all pages and image XObjects, converts images to RGB, optionally scales them, then replaces the originals with compressed versions.

OptimizationService — Helper Methods

- compressPageImage(): iterates PDResources XObjectNames, skips non-image or tiny objects
- toRgb(): converts ARGB/CMYK BufferedImage to TYPE_INT_RGB for JPEG compatibility
- scaleIfNeeded(): proportional downscale using BILINEAR interpolation if image exceeds 1200x1600

Known Bug

clamp(imageQuality, 0.1f, 0.1f) has min and max both set to 0.1f — the client quality parameter is always ignored. Fix: change to clamp(imageQuality, 0.1f, 0.9f).

OptimizationController — Endpoint

Property	Details
POST /pdf/optimize/compress	Accepts file + imageQuality float (default 0.6). Logs filename and quality via SLF4J.

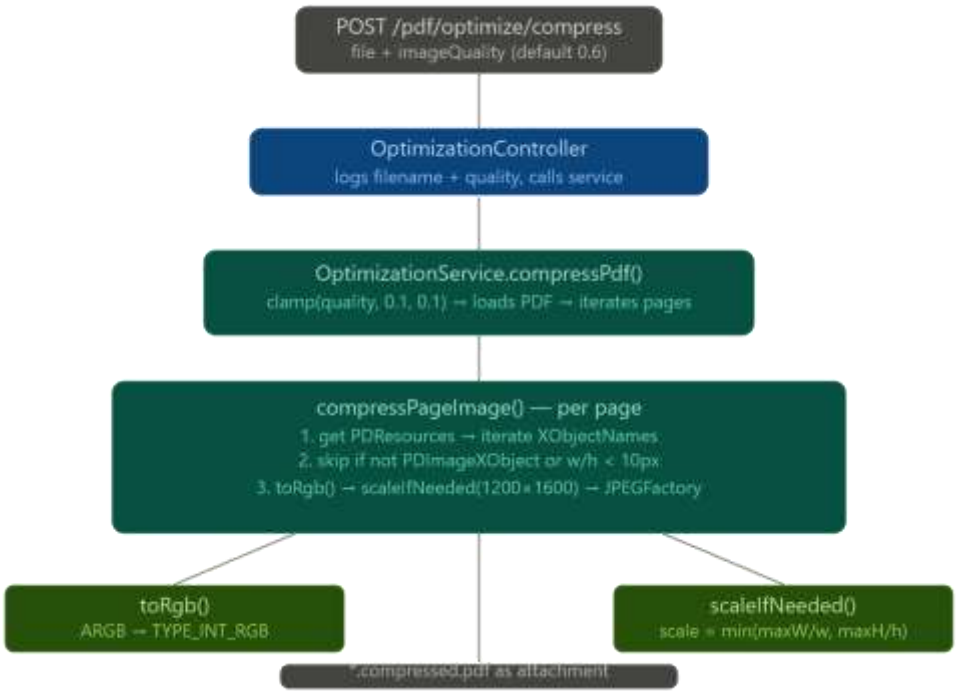

Figure 4: OptimizationController — Compression Flow

6.5 PDF Security — PDFSecurityService & PDFSecurityController

Description

PDFSecurityService applies and removes AES-256 password encryption using PDFBox's StandardProtectionPolicy. The lock operation configures fine-grained permissions. The unlock operation strips all security metadata and handles incorrect password exceptions.

Lock Operation

- Validates userPassword is not blank; ownerPassword falls back to userPassword if empty
- AccessPermission: canPrint(allowPrinting), canExtractContent(allowCopying), canModify(false)
- StandardProtectionPolicy with setEncryptionKeyLength(256) — AES-256

Unlock Operation

- Loads PDF with PDDocument.load(file, password) — decrypts on load
- setAllSecurityToBeRemoved(true) strips all encryption metadata before saving
- InvalidPasswordException caught and rethrown as CustomException('Incorrect Password')

PDFSecurityController — Endpoints

Property	Details
POST /pdf/security/lock	file + userPassword + ownerPassword + allowPrinting + allowCopying
POST /pdf/security/unlock	file + userPassword

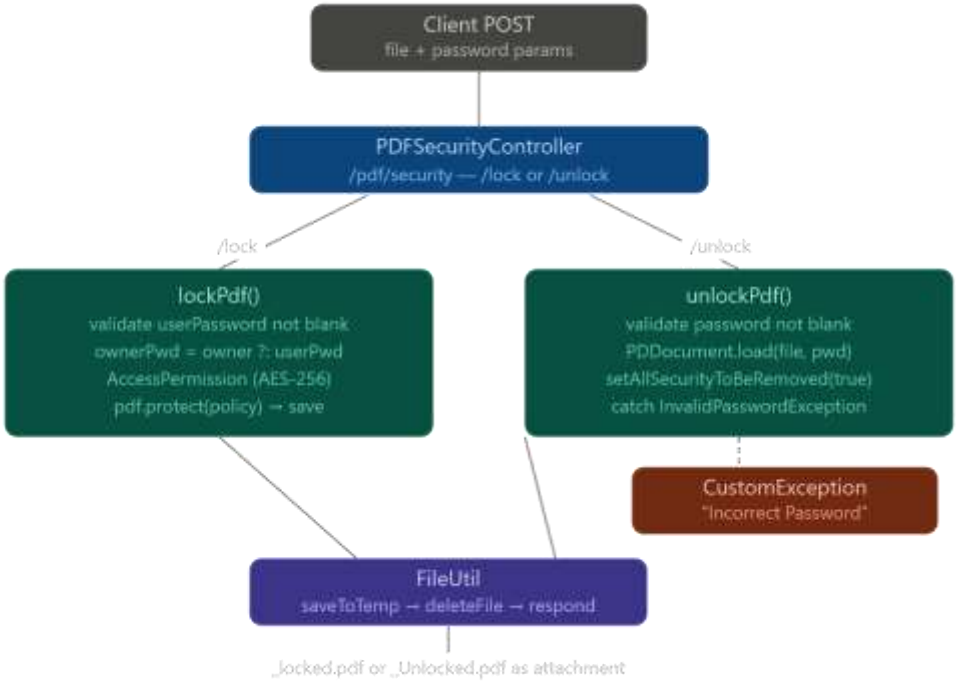

Figure 5: PDFSecurityController — Lock and Unlock Flow

6.6 Exception Handling — CustomException & GlobalException

CustomException is an unchecked RuntimeException subclass used throughout the application to signal business logic errors — invalid file inputs, wrong passwords, unsupported formats, or rule violations. GlobalException is annotated @RestControllerAdvice and intercepts all exceptions globally.

Property	Details
CustomException	extends RuntimeException — no throws declaration needed. Used in FileUtil, all services, and PDFController.
handleCustom()	@ExceptionHandler(CustomException.class) — returns HTTP 400 with message as plain-text body
handleException()	Returns HTTP 500 — BUG: missing @ExceptionHandler(Exception.class) annotation, so this method is never invoked

6.7 FileUtil — Shared Utility Class

Description

FileUtil is a Spring @Component shared by all four service classes. It centralizes all file I/O: PDF magic-byte validation, UUID-named temp file creation, silent deletion after processing, and HTTP download response construction.

Property	Details
validatePdf()	Reads first 4 bytes — checks for %PDF magic (0x25 0x50 0x44 0x46). Throws CustomException if invalid.
saveToTemp()	Generates UUID filename, resolves against temp dir, calls file.transferTo(). Returns Path.

deleteFile()	Files.deleteIfExists() in try-catch — failure silently swallowed to avoid masking the response.
downloadResponse()	ResponseEntity<byte[]> with 200 OK + Content-Disposition: attachment + Content-Length.
baseName()	Strips extension using lastIndexOf('.') — returns 'file' if input is null or empty.

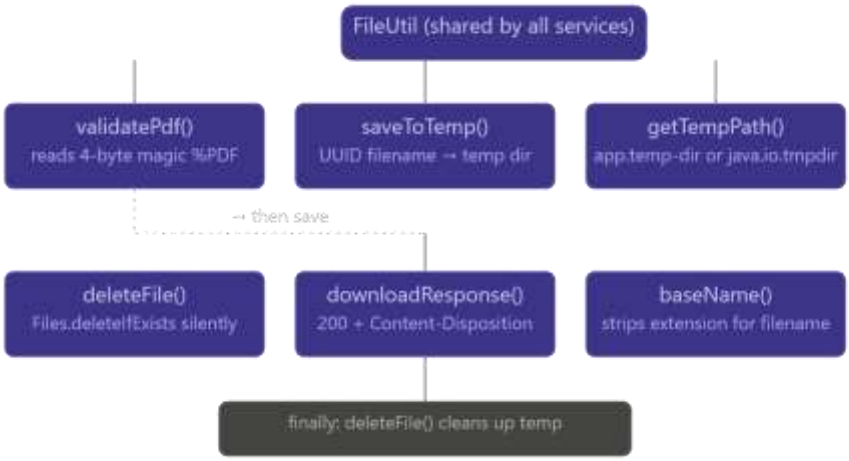

Figure 6: FileUtil — Shared Utility Method Lifecycle

7. Frontend Modules

The React frontend is organized into Pages, Shared Components, and an API Service Layer. All tool pages share a single ToolPage component for state and submission logic, keeping individual page files thin and maintainable.

7.1 Frontend Architecture

Description

The frontend follows a component-based architecture built with React and Vite. Pages map to individual PDF tools. Shared components handle file upload, loading feedback, and navigation. The api.js service layer manages all HTTP communication with the Spring Boot backend via Axios.

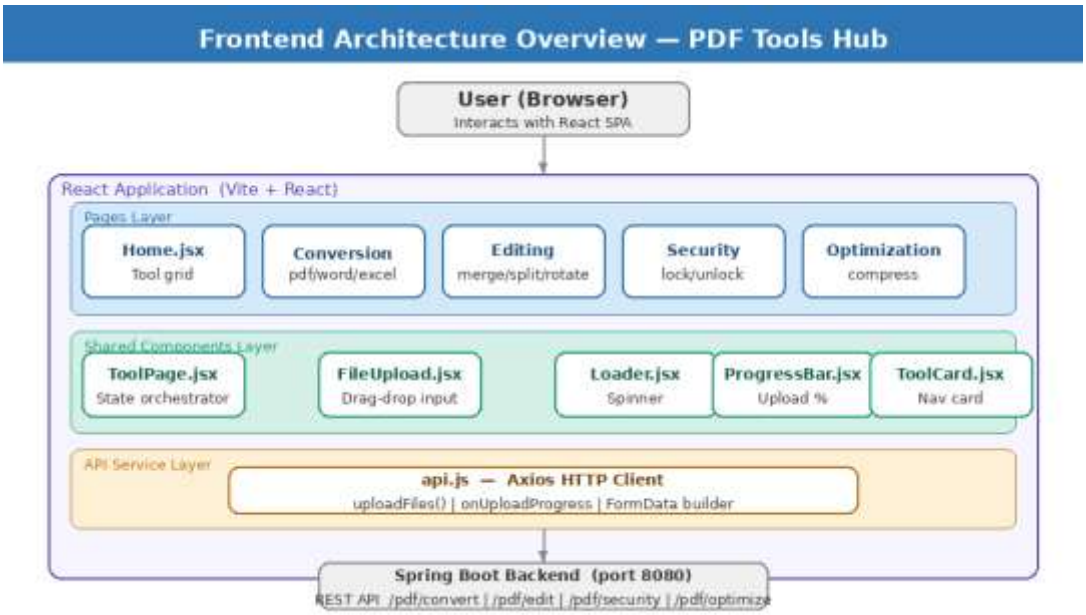

Figure 7: Frontend Architecture — Layer Overview

Property	Details
Pages Layer	One JSX file per tool — each renders <ToolPage title='...' endpoint='...' /> with no logic of its own
Shared Components	ToolPage.jsx (state owner), FileUpload.jsx (picker + preview), Loader, ProgressBar, ToolCard, NavBar
API Service	api.js — single Axios function: builds FormData, POSTs to backend, tracks upload progress
React Router v6	URL paths (/merge, /lock etc.) mapped to page components in App.jsx

7.2 Component Structure

The src/ directory is organized into three primary folders. This structure means adding a new tool requires only a new page file and a route entry — no other changes needed.

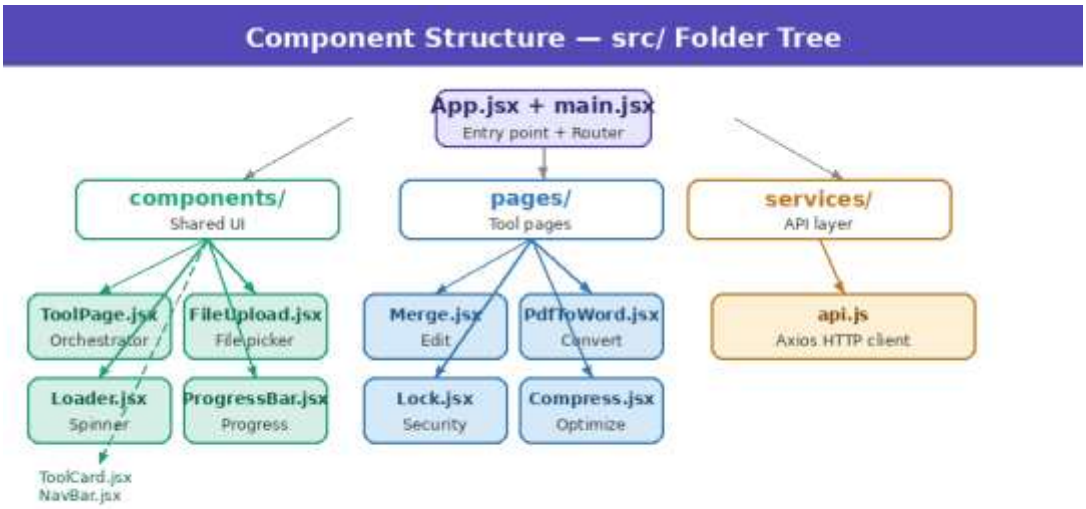

Figure 8: Component Folder Tree and Dependencies

7.3 API Service Layer — api.js

Description

api.js exports a single function uploadFiles(endpoint, files, setProgress). It constructs a FormData object, appends selected files under the key 'file', and sends a multipart POST via Axios. The onUploadProgress callback computes the upload percentage and drives the ProgressBar.

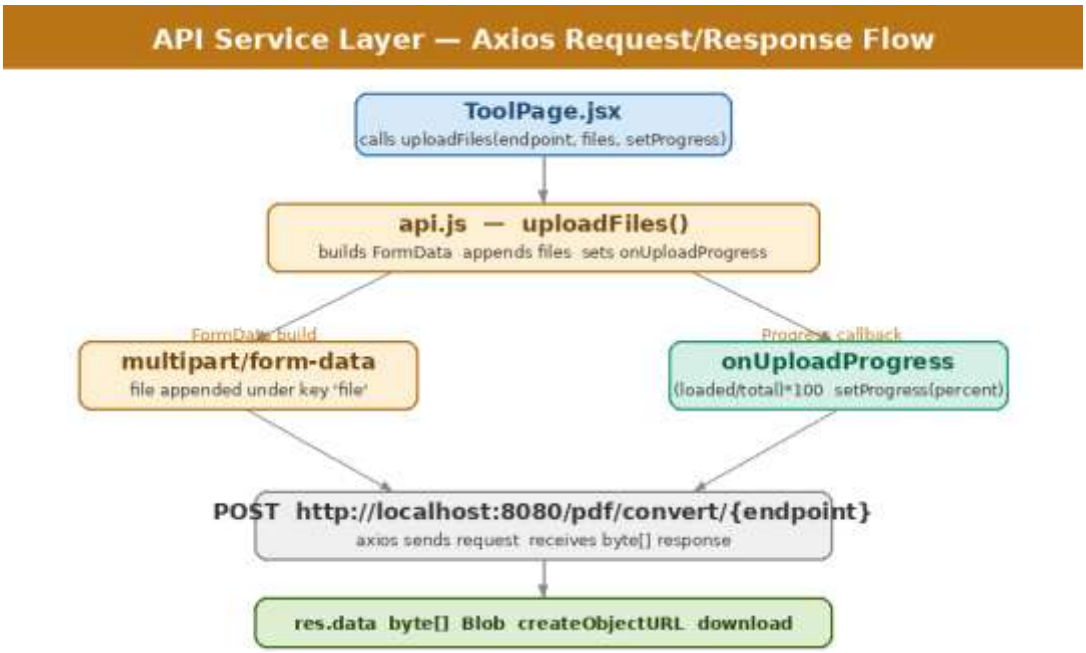

Figure 9: API Service — Axios Request and Response Flow

Known Bug — Hardcoded Base URL & Download Failure

api.js hardcodes const API = 'http://localhost:8080/pdf/convert' — tools under /pdf/edit, /pdf/security, and /pdf/optimize all receive 404 responses. Additionally, ToolPage.jsx calls window.open(res.fileUrl) which always fails because the backend returns raw bytes, not JSON. Fix: pass the full URL per endpoint, and use URL.createObjectURL(new Blob([res.data])) for download.

7.4 FileUpload Component

FileUpload renders a styled label as a click and drag-drop zone. When files are selected it calls setFiles() to pass the array up to ToolPage, and creates browser object URLs via URL.createObjectURL() to render live iframe previews of each selected PDF.

Property	Details
File input	Hidden <input type='file' multiple> inside a styled label
setFiles(files)	Passes File array to ToolPage state via prop callback
Object URL preview	URL.createObjectURL() generates temporary URL for each file — rendered in 120x140px iframes
Multiple files	Supports multi-selection — required for the Merge PDF operation

7.5 PDF Tool Pages

Every individual tool is a thin wrapper that renders a single shared component. The endpoint prop tells ToolPage which backend path to call.

Page	Category	Endpoint Prop	Backend Path
PdfToWord.jsx	Conversion	pdf-to-word	/pdf/convert/pdf-to-word
WordToPdf.jsx	Conversion	word-to-pdf	/pdf/convert/word-to-pdf
ExcelToPdf.jsx	Conversion	excel-to-pdf	/pdf/convert/excel-to-pdf
Merge.jsx	Editing	merge	/pdf/edit/merge
Split.jsx	Editing	split	/pdf/edit/split
Rotate.jsx	Editing	rotate	/pdf/edit/rotate
Lock.jsx	Security	lock	/pdf/security/lock
Unlock.jsx	Security	unlock	/pdf/security/unlock
Compress.jsx	Optimization	compress	/pdf/optimize/compress

7.6 User Interaction Flow

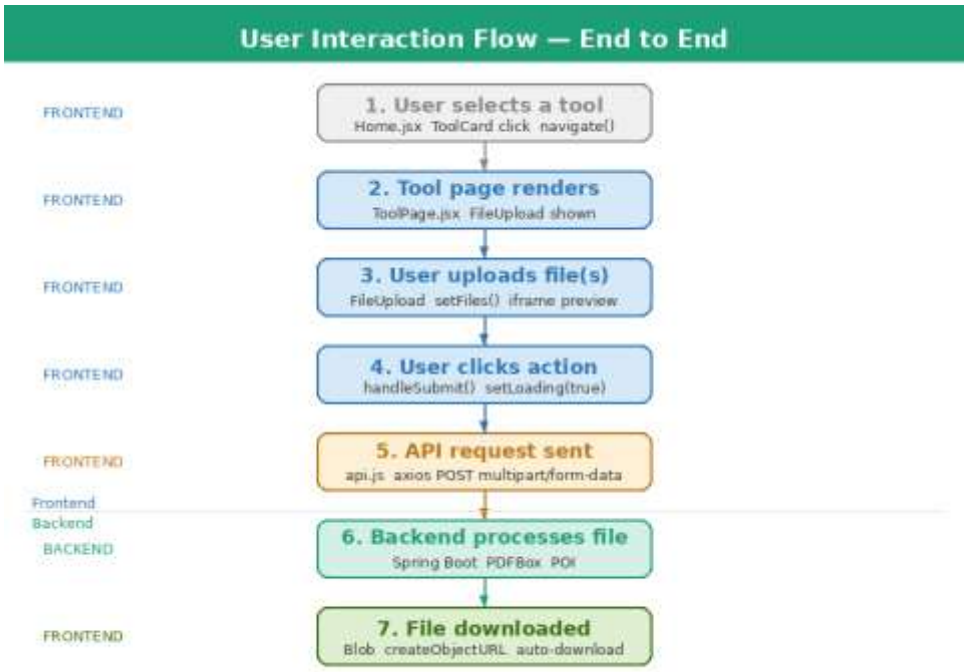

Figure 10: End-to-End User Interaction Flow

7.7 State Management

All state lives in ToolPage.jsx — the single state owner for every tool page. State is passed as props to child components; no external state library is used.

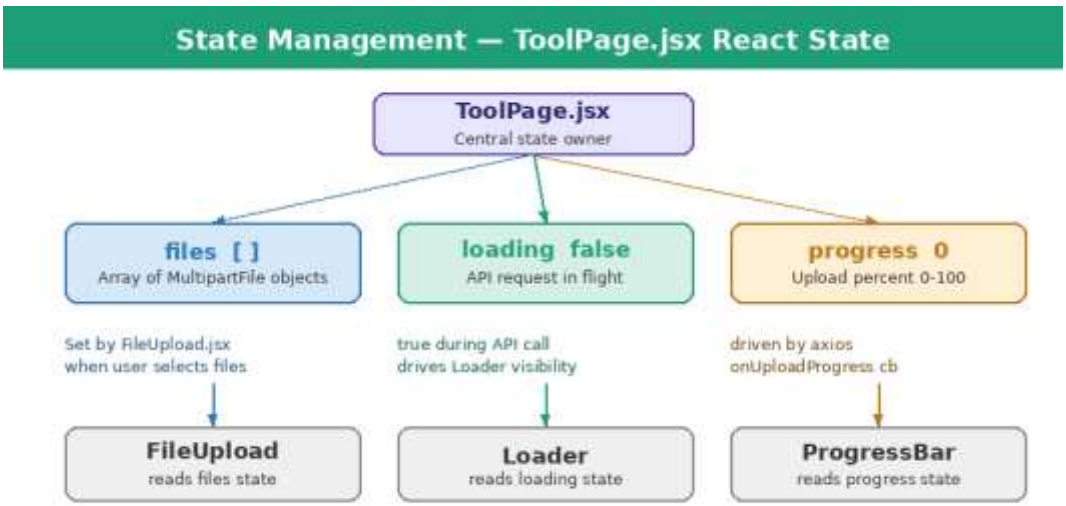

Figure 11: State Management — ToolPage.jsx as Central State Owner

Property	Details
files [ ]	File array set by FileUpload. Passed to api.js on submit.

loading false	Toggled true during API call. Controls Loader and ProgressBar visibility.
progress 0	Integer 0–100 driven by onUploadProgress. Animates the ProgressBar.

7.8 File Download Handling

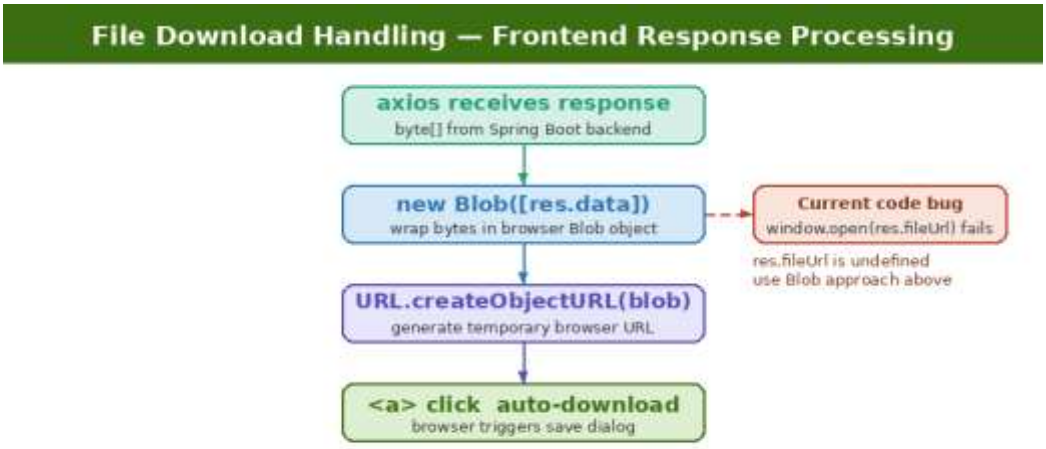

Figure 12: File Download Flow — Correct Blob Approach

7.9 Error Handling

The frontend provides basic error feedback through browser alert() dialogs. When a submission is attempted with no files selected, or when the API call fails for any reason, the user receives a plain alert. The loading state is always reset in the catch block to prevent a stuck spinner.

Property	Details
Empty file check	if (files.length === 0) — alert shown before API call is attempted
API errors	try/catch wraps uploadFiles() — catch calls alert('Error processing file')
Loading reset	setLoading(false) runs after both success and failure
Backend 400 errors	Axios throws on non-2xx responses — caught by the generic catch block

8. Challenges & Solutions

Property	Details
Large File Handling	Temporary UUID-named files written to disk; cleaned up in finally blocks to prevent storage buildup
Frontend-Backend Integration	Axios with multipart/form-data; CorsConfig allows all origins; consistent error propagation via CustomException
UI State Management	Centralized state in ToolPage.jsx — files, loading, and progress kept in one component

Cross-Browser Compatibility	React Bootstrap components ensure consistent rendering across Chrome, Edge, and Firefox
File Validation & Security	Server-side %PDF magic-byte check in FileUtil prevents non-PDF files from being processed

9. Future Enhancements

User Experience

- Drag-and-drop upload with visual feedback — HTML5 drag events on FileUpload zone
- Real-time progress bar for file upload and backend processing
- PDF file preview using react-pdf instead of plain iframes
- Toast notifications to replace browser alert() dialogs

Functionality

- User authentication — JWT-based login and session management
- File history dashboard — list of previous operations with re-download links
- Cloud storage integration — save outputs directly to Google Drive or AWS S3
- Batch processing — queue multiple files for a single operation
- AI-based PDF features — summarization, keyword extraction, and smart search

10. Conclusion

PDF Tools Hub successfully demonstrates the integration of a modern React frontend with a robust Spring Boot backend. The application provides a seamless platform for performing diverse PDF operations through a clean, user-friendly interface built with React Bootstrap.

The project covers core full-stack concepts including REST API design, multipart file handling, PDFBox and Apache POI document processing, layered backend architecture (Controller → Service → Utility), and React component state management. The modular design ensures each new PDF feature can be added with minimal changes — a new endpoint, a new service method, and a thin page wrapper.

This application not only simplifies everyday PDF tasks but also demonstrates production-level patterns in both frontend and backend development, making it a strong showcase of full-stack engineering capability.